

ted Question Bank

★ CONFIRMED ASKED	◆ HIGH PROBABILITY	● RELEVANT PATTERN	
35 Total Problems	8 Confirmed Asked	15 High Probability	12 Relevant Patterns

SECTION 1: CONFIRMED ASKED AT SYNECHRON

Questions verified from real Synchroon campus placement candidate reports (2023–2025)

Q1

First Non-Repeating Character in a String

Easy ★ CONFIRMED ASKED AT SYNECHRON

Given a string, find and return the first character that does not repeat anywhere in the string. If all characters repeat, return '_'. This is the single most commonly reported Synchroon Java test question.

INPUT	OUTPUT
Input: "aabbcdce"	Output: 'c'
Input: "aabb"	Output: '_'
Input: "stress"	Output: 't'

Approach 1 — LinkedHashMap (Best for interviews)

Use a LinkedHashMap to store each character's frequency while preserving insertion order. Then iterate through the map to find the first entry with frequency 1.

```
import java.util.*;

public class FirstNonRepeating {

    public static char firstNonRepeating(String s) {
        // LinkedHashMap preserves insertion order
        LinkedHashMap<Character, Integer> freq = new LinkedHashMap<>();

        // Count frequency of each character
        for (char c : s.toCharArray()) {
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        }

        // Find first character with frequency 1
        for (Map.Entry<Character, Integer> entry : freq.entrySet()) {
            if (entry.getValue() == 1) return entry.getKey();
        }

        return '_'; // no non-repeating character found
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        System.out.println(firstNonRepeating(s));
        sc.close();
    }
}
```

Time: O(n)

Space: O(1) — max 256
chars

Approach: LinkedHashMap frequency count

Approach 2 — int[256] array (faster, less memory)

```

public static char firstNonRepeatingFast(String s) {
    int[] freq = new int[256]; // ASCII array
    for (char c : s.toCharArray()) freq[c]++;
    for (char c : s.toCharArray()) {
        if (freq[c] == 1) return c;
    }
    return '_';
}
// Two passes: first to count, second to find first with count==1

```

TIP: LinkedHashMap approach is preferred in interviews because it clearly shows your understanding of ordered maps. The int[256] approach is faster but less readable.

Q2

Palindrome Check — String & Number

Easy ★ CONFIRMED ASKED AT SYNECHRON

Check if a given string or number reads the same forwards and backwards. Synchro candidates reported both string palindrome and number palindrome variants.

INPUT	OUTPUT
Input: "racecar"	Output: true
Input: "hello"	Output: false
Input: 121	Output: true
Input: 123	Output: false

```

import java.util.Scanner;

public class Palindrome {
    // String palindrome – two pointer approach
    public static boolean isPalindromeString(String s) {
        s = s.toLowerCase().replaceAll("[^a-z0-9]", ""); // clean input
        int left = 0, right = s.length() - 1;
        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) return false;
            left++;
            right--;
        }
        return true;
    }

    // Number palindrome – without converting to String
    public static boolean isPalindromeNumber(int n) {
        if (n < 0) return false; // negative numbers not palindromes
        if (n != 0 && n % 10 == 0) return false; // trailing zero case
        int original = n, reversed = 0;
        while (n > 0) {
            reversed = reversed * 10 + n % 10;
            n /= 10;
        }
    }
}

```

```
}  
return original == reversed;  
}  
  
// Shortcut: using StringBuilder (quick in coding tests)  
public static boolean isPalindromeShortcut(String s) {  
    String rev = new StringBuilder(s).reverse().toString();  
    return s.equals(rev);  
}  
  
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    String input = sc.next();  
    System.out.println(isPalindromeString(input));  
}  
}
```

Time: $O(n)$

Space: $O(1)$

Approach: Two-pointer / Number reversal

TIP: In the test, if given a string, use the two-pointer method. If given a number, use the digit-reversal method. Both should be in your muscle memory.

Q3

Reverse a String (Without Using reverse())

Easy ★ CONFIRMED ASKED AT SYNECHRON

Reverse a given string without using the built-in reverse() method of StringBuilder. Show you understand character arrays and two-pointer technique.

INPUT	OUTPUT
Input: "HackerRank"	Output: "knaRrekcaH"
Input: "Synechron"	Output: "norchenyS"

```
import java.util.Scanner;

public class ReverseString {
    // Method 1: Two-pointer with char array (best approach)
    public static String reverse(String s) {
        char[] chars = s.toCharArray();
        int left = 0, right = chars.length - 1;
        while (left < right) {
            char temp = chars[left];
            chars[left] = chars[right];
            chars[right] = temp;
            left++;
            right--;
        }
        return new String(chars);
    }

    // Method 2: Loop from end to start with StringBuilder
    public static String reverseLoop(String s) {
        StringBuilder sb = new StringBuilder();
        for (int i = s.length() - 1; i >= 0; i--) {
            sb.append(s.charAt(i));
        }
        return sb.toString();
    }

    // Method 3: Recursive reversal
    public static String reverseRecursive(String s) {
        if (s.isEmpty()) return s;
        return reverseRecursive(s.substring(1)) + s.charAt(0);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.next();
        System.out.println(reverse(s));
    }
}
```

Time: O(n)

Space: O(n)

Approach: Two-pointer char swap

Q4

Find All Pairs That Sum to Target

Easy ★ CONFIRMED ASKED AT SYNECHRON

Given a list of integers and a target sum, find all pairs of numbers that add up to the target. Synechron specifically reported: 'given a list, find pairs that sum to 6'.

INPUT	OUTPUT
Input: [1,2,3,4,5,6], target=6	Pairs: (1,5) (2,4)
Input: [1,5,3,3,2,4], target=6	Pairs: (1,5) (3,3) (2,4)

```
import java.util.*;

public class PairSum {

    // Method 1: HashSet – O(n) time, handles duplicates well
    public static void findPairs(int[] arr, int target) {
        HashSet<Integer> seen = new HashSet<>();
        HashSet<String> printed = new HashSet<>(); // avoid duplicate pairs
        for (int num : arr) {
            int complement = target - num;
            if (seen.contains(complement)) {
                // Create canonical pair (smaller, larger) to avoid duplicates
                int a = Math.min(num, complement);
                int b = Math.max(num, complement);
                String pair = a + "," + b;
                if (!printed.contains(pair)) {
                    System.out.println("(" + a + ", " + b + ")");
                    printed.add(pair);
                }
            }
            seen.add(num);
        }
    }

    // Method 2: Two pointer (requires sorted array)
    public static void findPairsSorted(int[] arr, int target) {
        Arrays.sort(arr);
        int left = 0, right = arr.length - 1;
        while (left < right) {
            int sum = arr[left] + arr[right];
            if (sum == target) {
                System.out.println("(" + arr[left] + ", " + arr[right] + ")");
                left++;
                right--;
            } else if (sum < target) left++;
            else right--;
        }
    }
}
```

```
}  
  
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    int[] arr = new int[n];  
    for (int i = 0; i < n; i++) arr[i] = sc.nextInt();  
    int target = sc.nextInt();  
    findPairs(arr, target);  
    sc.close();  
}  
}
```

Time: $O(n)$

Space: $O(n)$

Approach: HashSet complement lookup

Q5

Find N Largest Primes in a Range & Their Sum

Easy-Medium ★ CONFIRMED ASKED AT SYNECHRON

Given a range [start, end] and a number N, find the N largest prime numbers in that range and print their sum. Synchron reported: 'range 2 to 10, find 3 largest primes — answer: 3+5+7=15'.

INPUT	OUTPUT
Input: start=2, end=30, N=3 Input: start=2, end=10, N=3	Primes: 23,29,31->23+29+31? No: largest 3 from 2-30: 29,23,19 sum=71 Primes: 7,5,3 sum=15

```
import java.util.*;

public class PrimesInRange {
    // Check if a number is prime
    public static boolean isPrime(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false;
        for (int i = 3; i * i <= n; i += 2) {
            if (n % i == 0) return false;
        }
        return true;
    }

    // Find N largest primes in range and return their sum
    public static long nLargestPrimesSum(int start, int end, int n) {
        // Collect all primes in range
        ArrayList<Integer> primes = new ArrayList<>();
        for (int i = start; i <= end; i++) {
            if (isPrime(i)) primes.add(i);
        }

        // Sort descending to get N largest
        Collections.sort(primes, Collections.reverseOrder());

        System.out.print("Top " + n + " primes: ");
        long sum = 0;
        for (int i = 0; i < Math.min(n, primes.size()); i++) {
            System.out.print(primes.get(i) + " ");
            sum += primes.get(i);
        }
        System.out.println();
        return sum;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int start = sc.nextInt();
        int end = sc.nextInt();
    }
}
```



```

int n = sc.nextInt();
long sum = nLargestPrimesSum(start, end, n);
System.out.println("Sum = " + sum);
sc.close();
}
}
// For start=2, end=10, n=3:
// Primes in range: 2,3,5,7
// Top 3: 7, 5, 3 Sum = 15

```

Time: $O(n * \sqrt{n})$

Space: $O(n)$

Approach: isPrime check + sort

Q6

Shift Evens Left, Odds Right (In-Place Partition)

Easy-Medium ★ CONFIRMED ASKED AT SYNECHRON

Given an array, rearrange it so all even numbers come before all odd numbers. Do it in-place ($O(1)$ extra space). Synchrotron reported this with an 'optimise if possible' follow-up.

INPUT	OUTPUT
Input: [2,5,8,7,1,3,6,4,5,9]	Output: [2,8,6,4,5,7,1,3,5,9] (all evens left, all odds right)

```

import java.util.*;

public class EvenOddPartition {
    // Optimal two-pointer approach - O(n) time, O(1) space
    public static void partition(int[] arr) {
        int left = 0, right = arr.length - 1;
        while (left < right) {
            // Move left pointer right while it points to even
            while (left < right && arr[left] % 2 == 0) left++;
            // Move right pointer left while it points to odd
            while (left < right && arr[right] % 2 != 0) right--;
            // Swap: arr[left] is odd, arr[right] is even
            if (left < right) {
                int temp = arr[left];
                arr[left] = arr[right];
                arr[right] = temp;
                left++;
                right--;
            }
        }
    }

    // Alternative: Dutch National Flag style (stable order)
    public static int[] partitionStable(int[] arr) {
        ArrayList<Integer> evens = new ArrayList<>();

```

```
ArrayList<Integer> odds = new ArrayList<>();
for (int x : arr) {
    if (x % 2 == 0) evens.add(x);
    else odds.add(x);
}
evens.addAll(odds);
return evens.stream().mapToInt(Integer::intValue).toArray();
}

public static void main(String[] args) {
    int[] arr = {2, 5, 8, 7, 1, 3, 6, 4, 5, 9};
    partition(arr);
    System.out.println(Arrays.toString(arr));
}
}
```

Time: $O(n)$

Space: $O(1)$

Approach: Two-pointer swap

TIP: If asked to 'optimise', say: two-pointer is $O(n)$ time and $O(1)$ space — optimal. No sorting needed.

Q7

Runtime Polymorphism — Which Method Gets Called?

Easy ★ CONFIRMED ASKED AT SYNECHRON

Synechron directly asked: If parent and child both have the same method, and you create a parent reference pointing to a child object — which method gets called? Write a program demonstrating this.

```
// QUESTION: What is the output of this program?
class Animal {
    public void sound() {
        System.out.println("Animal makes a sound");
    }
    public void breathe() {
        System.out.println("Animal breathes");
    }
}

class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks"); // overrides parent
    }
    public void fetch() {
        System.out.println("Dog fetches"); // Dog-specific
    }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        // Parent reference, Child object – RUNTIME POLYMORPHISM
        Animal a = new Dog();

        a.sound(); // OUTPUT: "Dog barks" <- overridden method called
        a.breathe(); // OUTPUT: "Animal breathes" <- not overridden
        // a.fetch(); // COMPILER ERROR – Animal ref can't see Dog methods
        // To call Dog's fetch(), downcast:
        ((Dog) a).fetch(); // OUTPUT: "Dog fetches"

        // The decision is made at RUNTIME based on actual object type
        // This is called Dynamic Method Dispatch
        // instanceof check before casting
        if (a instanceof Dog) {
            Dog d = (Dog) a;
            d.fetch();
        }
    }
}
```

Scenario	Method Called	Why
Animal a = new Dog(); a.sound()	Dog's sound()	Overridden — runtime dispatch

Animal a = new Dog(); a.breathe()	Animal's breathe()	Not overridden in Dog
Animal a = new Animal(); a.sound()	Animal's sound()	Actual object is Animal
Dog d = new Dog(); d.sound()	Dog's sound()	Direct call on Dog object

NOTE: Rule: Java always calls the method of the ACTUAL object (runtime type), not the reference type. This is runtime polymorphism / dynamic dispatch.

Q8

Diamond Problem in Java & How Interfaces Solve It

Easy ★ CONFIRMED ASKED AT SYNECHRON

Synechron directly asked about the diamond problem. Java does not support multiple class inheritance to avoid this. Interfaces are the solution — know the explanation AND the code.

```
// THE DIAMOND PROBLEM:
// Class A has method display()
// Class B extends A, overrides display()
// Class C extends A, overrides display()
// If class D extends both B and C – which display() does D inherit? AMBIGUITY!
// Java SOLVES this by NOT allowing a class to extend multiple classes.

// WRONG – does NOT compile in Java:
// class D extends B, C { } // COMPILE ERROR: multiple inheritance not allowed

// SOLUTION 1: Use Interfaces
interface Printable {
    void print();
}

interface Displayable {
    void display();
}

// A class CAN implement multiple interfaces
class MultiImpl implements Printable, Displayable {
    public void print() { System.out.println("Printing..."); }
    public void display() { System.out.println("Displaying..."); }
}

// SOLUTION 2: Java 8 default method conflict – resolved by override
interface A { default void greet() { System.out.println("Hello from A"); } }
interface B { default void greet() { System.out.println("Hello from B"); } }

class C implements A, B {
    // MUST override greet() when two interfaces have same default method
    @Override
    public void greet() {
        A.super.greet(); // explicitly choose which interface's default to call
        System.out.println("Hello from C");
    }
}
```

```
public class DiamondDemo {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.greet();  
        // Output: Hello from A  
        // Hello from C  
    }  
}
```

NOTE: Key answer: Java does not allow multiple class inheritance to avoid the diamond problem ambiguity. Interfaces are used instead. From Java 8, if two interfaces have the same default method, the implementing class **MUST** override it.

SECTION 2: HIGH PROBABILITY PROBLEMS

Very likely to appear based on Synechron's pattern and BFSI/fintech focus

Q9

Check if Two Strings Are Anagrams

Easy

Two strings are anagrams if they contain the same characters in the same frequency (order doesn't matter). This is a standard string manipulation problem reported at many fintech companies.

INPUT	OUTPUT
Input: "listen", "silent"	Output: true
Input: "hello", "world"	Output: false

```
import java.util.*;

public class Anagram {
    // Method 1: Sort both strings and compare
    public static boolean isAnagramSort(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        char[] a = s1.toLowerCase().toCharArray();
        char[] b = s2.toLowerCase().toCharArray();
        Arrays.sort(a);
        Arrays.sort(b);
        return Arrays.equals(a, b);
    }

    // Method 2: Frequency count – O(n), better approach
    public static boolean isAnagram(String s1, String s2) {
        if (s1.length() != s2.length()) return false;
        int[] count = new int[256];
        for (char c : s1.toLowerCase().toCharArray()) count[c]++;
        for (char c : s2.toLowerCase().toCharArray()) count[c]--;
        for (int val : count) {
            if (val != 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s1 = sc.next(), s2 = sc.next();
        System.out.println(isAnagram(s1, s2));
    }
}
```

Time: O(n)

Space: O(1)

Approach: Frequency array count

Q10

Print Common (Duplicate) Characters Between Two Strings

Easy ★ CONFIRMED ASKED AT SYNECHRON

Synechron reported: 'given two strings, print duplicate (common) characters between them.' Find characters present in both strings.

INPUT	OUTPUT
Input: "Java With Collections" "Java Selenium"	Common chars: J,a,v,a,e,l,i

```
import java.util.*;

public class CommonChars {

    public static void findCommon(String s1, String s2) {
        HashSet<Character> set1 = new HashSet<>();
        HashSet<Character> common = new HashSet<>();

        // Add all chars from s1
        for (char c : s1.toLowerCase().toCharArray()) {
            if (c != ' ') set1.add(c);
        }

        // Check s2 chars against set1
        for (char c : s2.toLowerCase().toCharArray()) {
            if (c != ' ' && set1.contains(c)) {
                common.add(c);
            }
        }

        // Sort and print
        TreeSet<Character> sorted = new TreeSet<>(common);
        System.out.println("Common characters: " + sorted);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s1 = sc.nextLine();
        String s2 = sc.nextLine();
        findCommon(s1, s2);
    }
}
```

Time: $O(n + m)$

Space: $O(n)$

Approach: HashSet intersection

Q11

Longest Substring Without Repeating Characters

Medium

Find the length of the longest substring that has no repeating characters. Synecron and other fintech companies frequently ask sliding window problems in this format.

INPUT	OUTPUT
Input: "abcabcbb"	Output: 3 (abc)
Input: "bbbbbb"	Output: 1 (b)
Input: "pwwkew"	Output: 3 (wke)

```
import java.util.*;

public class LongestSubstring {
    // Sliding Window with HashMap — O(n)
    public static int lengthOfLongestSubstring(String s) {
        HashMap<Character, Integer> map = new HashMap<>();
        int maxLen = 0;
        int left = 0; // left boundary of window
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            // If char seen before and is inside current window
            if (map.containsKey(c) && map.get(c) >= left) {
                left = map.get(c) + 1; // shrink window
            }
            map.put(c, right); // store/update char's latest index
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }

    // Also print the substring
    public static String longestSubstringStr(String s) {
        HashMap<Character, Integer> map = new HashMap<>();
        int maxLen = 0, start = 0, left = 0;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (map.containsKey(c) && map.get(c) >= left) {
                left = map.get(c) + 1;
            }
            map.put(c, right);
            if (right - left + 1 > maxLen) {
                maxLen = right - left + 1;
                start = left;
            }
        }
        return s.substring(start, start + maxLen);
    }
}
```



```

}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String s = sc.next();
    System.out.println(lengthOfLongestSubstring(s));
    System.out.println(longestSubstringStr(s));
}
}

```

Time: $O(n)$

Space: $O(\min(n, 256))$

Approach: Sliding window + HashMap

Q12

Find Second Largest Element in Array

Easy

A classic array problem. Find the second largest distinct element in one pass. Reported at Synchroon in both direct and variation forms.

INPUT	OUTPUT
Input: [12, 35, 1, 10, 34, 1]	Output: 34
Input: [5, 5, 5]	Output: -1 (no second distinct)

```

public class SecondLargest {
    public static int secondLargest(int[] arr) {
        int first = Integer.MIN_VALUE;
        int second = Integer.MIN_VALUE;
        for (int num : arr) {
            if (num > first) {
                second = first;
                first = num;
            } else if (num > second && num != first) {
                // num is less than first but greater than second
                // and not equal to first (we want DISTINCT)
                second = num;
            }
        }
        return (second == Integer.MIN_VALUE) ? -1 : second;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        System.out.println(secondLargest(arr));
    }
}

```

Time: $O(n)$	Space: $O(1)$	Approach: Single pass with two variables
--------------	---------------	--

Q13

FizzBuzz — Classic Warm-Up

Easy

Print numbers 1 to N. For multiples of 3 print 'Fizz', multiples of 5 print 'Buzz', multiples of both print 'FizzBuzz'. Universally asked at every company as a warm-up.

```
import java.util.Scanner;

public class FizzBuzz {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for (int i = 1; i <= n; i++) {
            // Check % 15 FIRST (both 3 and 5) — common mistake to forget
            if (i % 15 == 0) System.out.println("FizzBuzz");
            else if (i % 3 == 0) System.out.println("Fizz");
            else if (i % 5 == 0) System.out.println("Buzz");
            else System.out.println(i);
        }
        sc.close();
    }
}
```

Time: O(n)

Space: O(1)

Approach: Simple iteration

WARNING: Always check % 15 before % 3 and % 5. Checking % 3 first will catch multiples of 15 as just 'Fizz' — wrong!

Q14

Character / Word Frequency Count

Easy

Count frequency of each character or word in a string. Sort by frequency descending. Extremely common at Synchroon — tests HashMap mastery.

INPUT	OUTPUT
Input: "banana"	a: 3 n: 2 b: 1

```
import java.util.*;

public class FrequencyCount {
    // Character frequency — sorted by count descending
    public static void charFrequency(String s) {
        HashMap<Character, Integer> freq = new HashMap<>();
    }
}
```

```

for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}

// Sort by value descending, then key ascending
List<Map.Entry<Character, Integer>> list = new ArrayList<>(freq.entrySet());
list.sort((a, b) -> b.getValue() != a.getValue() ?
    b.getValue() - a.getValue() :
    a.getKey() - b.getKey());

for (Map.Entry<Character, Integer> e : list) {
    System.out.println(e.getKey() + ": " + e.getValue());
}
}

// Word frequency in a sentence
public static void wordFrequency(String sentence) {
    LinkedHashMap<String, Integer> freq = new LinkedHashMap<>();
    for (String word : sentence.toLowerCase().split("\\s+")) {
        freq.put(word, freq.getOrDefault(word, 0) + 1);
    }
    freq.forEach((k, v) -> System.out.println(k + ": " + v));
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String s = sc.nextLine();
    charFrequency(s);
}
}

```

Time: $O(n \log n)$

Space: $O(n)$

Approach: HashMap + sort by value

Q15

OOP — Student Grade System (Full Program)

Easy-Medium

A complete OOP problem combining class design, methods, and formatted output. Synechron tests typically include at least one OOP design question.

INPUT	OUTPUT
3 Alice 85 90 78 Bob 55 60 45 Charlie 95 98 100	Alice Avg:84.33 Grade:B Top:90 Bob Avg:53.33 Grade:F Top:60 Charlie Avg:97.67 Grade:A Top:100

```
import java.util.*;

class Student {
    String name;
    int[] marks;

    Student(String name, int[] marks) {
        this.name = name;
        this.marks = marks;
    }

    double average() {
        int sum = 0;
        for (int m : marks) sum += m;
        return (double) sum / marks.length;
    }

    char grade() {
        double avg = average();
        if (avg >= 90) return 'A';
        if (avg >= 80) return 'B';
        if (avg >= 70) return 'C';
        if (avg >= 60) return 'D';
        return 'F';
    }

    int topScore() {
        int max = marks[0];
        for (int m : marks) if (m > max) max = m;
        return max;
    }
}

public class StudentGrade {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(); sc.nextLine();
        for (int i = 0; i < n; i++) {
            String[] parts = sc.nextLine().trim().split("\\s+");
            String name = parts[0];
```

```

int[] marks = new int[parts.length - 1];
for (int j = 1; j < parts.length; j++)
marks[j-1] = Integer.parseInt(parts[j]);
Student s = new Student(name, marks);
System.out.printf("%-10s Avg:%.2f Grade:%c Top:%d%n",
s.name, s.average(), s.grade(), s.topScore());
}
sc.close();
}
}

```

Time: $O(n * m)$

Space: $O(n)$

Approach: OOP design

Q16

String to Integer Without parseInt()

Easy-Medium

Convert a string of digits to an integer manually, without using `Integer.parseInt()`. Tests understanding of char arithmetic and ASCII values.

INPUT	OUTPUT
Input: "12345"	Output: 12345
Input: "-678"	Output: -678
Input: "0042"	Output: 42

```

public class StringToInt {
    public static int convert(String s) {
        s = s.trim();
        if (s.isEmpty()) return 0;
        int sign = 1, i = 0;
        if (s.charAt(0) == '-') { sign = -1; i = 1; }
        else if (s.charAt(0) == '+') { i = 1; }
        int result = 0;
        while (i < s.length()) {
            char c = s.charAt(i);
            if (c < '0' || c > '9') break; // non-digit: stop
            result = result * 10 + (c - '0'); // 'c'-'0' gives digit value
            i++;
        }
        return sign * result;
    }
    public static void main(String[] args) {
        System.out.println(convert("12345")); // 12345
        System.out.println(convert("-678")); // -678
        System.out.println(convert(" 42 ")); // 42
    }
}

```

```
}  
// KEY: char '5' - char '0' = integer 5 (ASCII arithmetic)
```

Time: $O(n)$

Space: $O(1)$

Approach: ASCII char arithmetic

Q17

Fibonacci Series & Check if Number is Fibonacci

Easy

Print first N Fibonacci numbers, or check if a given number is a Fibonacci number. Both variants are commonly seen in fresher Java tests.

```
import java.util.Scanner;

public class Fibonacci {

    // Print first N Fibonacci numbers – iterative O(n)
    public static void printFib(int n) {
        long a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            long temp = a + b;
            a = b;
            b = temp;
        }
        System.out.println();
    }

    // Get Nth Fibonacci – iterative
    public static long nthFib(int n) {
        if (n <= 1) return n;
        long a = 0, b = 1;
        for (int i = 2; i <= n; i++) {
            long c = a + b; a = b; b = c;
        }
        return b;
    }

    // Check if number is a perfect square (helper)
    private static boolean isPerfectSquare(long x) {
        long s = (long) Math.sqrt(x);
        return s * s == x;
    }

    // A number n is Fibonacci if (5n^2 + 4) or (5n^2 - 4) is a perfect square
    public static boolean isFibonacci(long n) {
        return isPerfectSquare(5 * n * n + 4) ||
            isPerfectSquare(5 * n * n - 4);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        printFib(n); // e.g. n=8: 0 1 1 2 3 5 8 13
        System.out.println(nthFib(10)); // 55
        System.out.println(isFibonacci(13)); // true
        System.out.println(isFibonacci(14)); // false
    }
}
```



```
}  
}
```

Time: O(n)

Space: O(1)

Approach: Iterative — 3 variables

Q18

Balanced Parentheses / Brackets

Easy-Medium

Check if a given string of brackets is balanced. Every opening bracket must have a corresponding closing bracket in the correct order. Classic Stack problem.

INPUT	OUTPUT
Input: "{[()]}"	Output: true
Input: "{[(])}"	Output: false
Input: "((("	Output: false

```
import java.util.*;  
  
public class BalancedBrackets {  
    public static boolean isBalanced(String s) {  
        Deque<Character> stack = new ArrayDeque<>();  
        for (char c : s.toCharArray()) {  
            // Push opening brackets  
            if (c == '(' || c == '[' || c == '{') {  
                stack.push(c);  
            }  
            // Check closing brackets  
            else if (c == ')' || c == ']' || c == '}') {  
                if (stack.isEmpty()) return false; // nothing to match  
                char top = stack.pop();  
                if (c == ')' && top != '(') return false;  
                if (c == ']' && top != '[') return false;  
                if (c == '}' && top != '{') return false;  
            }  
        }  
        return stack.isEmpty(); // all brackets matched  
    }  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        String s = sc.next();  
        System.out.println(isBalanced(s));  
    }  
}
```

Time: O(n)

Space: O(n)

Approach: Stack push/pop

Q19

Remove Duplicates from Array / Find All Non-Duplicates

Easy

Two variants: (1) remove all duplicates and return unique elements sorted, (2) find elements that appear exactly once. Both reported at fintech company HackerRank tests.

INPUT	OUTPUT
Input: [1, 2, 3, 2, 4, 1, 5] Variant 2: find non-duplicates	Unique sorted: [1,2,3,4,5] Non-duplicates: [3,4,5]

```
import java.util.*;

public class Duplicates {

    // Remove duplicates – keep unique elements in sorted order
    public static int[] removeDuplicates(int[] arr) {
        TreeSet<Integer> set = new TreeSet<>();
        for (int x : arr) set.add(x);
        return set.stream().mapToInt(Integer::intValue).toArray();
    }

    // Find elements that appear exactly once
    public static void findNonDuplicates(int[] arr) {
        HashMap<Integer, Integer> freq = new HashMap<>();
        for (int x : arr) freq.put(x, freq.getOrDefault(x, 0) + 1);
        List<Integer> result = new ArrayList<>();
        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            if (e.getValue() == 1) result.add(e.getKey());
        }
        Collections.sort(result);
        System.out.println("Non-duplicates: " + result);
    }

    // Find all duplicate values (appear more than once)
    public static void findDuplicates(int[] arr) {
        HashSet<Integer> seen = new HashSet<>();
        HashSet<Integer> dups = new LinkedHashSet<>();
        for (int x : arr) {
            if (!seen.add(x)) dups.add(x);
        }
        System.out.println("Duplicates: " + dups);
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2, 4, 1, 5};
        System.out.println(Arrays.toString(removeDuplicates(arr)));
        findNonDuplicates(arr);
        findDuplicates(arr);
    }
}
```

Time: $O(n \log n)$

Space: $O(n)$

Approach: HashMap freq + TreeSet

Q20

Maximum Palindromic Number in a List

Easy-Medium ★ CONFIRMED ASKED AT SYNECHRON

Synechron directly reported: 'write a program for maximum palindromic number in a given list of numeric values'. Find the largest number that is also a palindrome.

INPUT	OUTPUT
Input: [121, 333, 456, 12321, 99, 100]	Output: 12321

```
import java.util.*;

public class MaxPalindromeNumber {

    public static boolean isPalindromeNum(int n) {
        if (n < 0) return false;
        String s = String.valueOf(n);
        int l = 0, r = s.length() - 1;
        while (l < r) {
            if (s.charAt(l) != s.charAt(r)) return false;
            l++; r--;
        }
        return true;
    }

    public static int maxPalindrome(int[] nums) {
        int maxPalin = -1;
        for (int n : nums) {
            if (isPalindromeNum(n) && n > maxPalin) {
                maxPalin = n;
            }
        }
        return maxPalin; // -1 if no palindrome found
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        System.out.println("Max palindrome: " + maxPalindrome(arr));
    }
}
```

Time: $O(n * d)$

Space: $O(1)$

Approach: d = digits per number

SECTION 3: RELEVANT PATTERN PROBLEMS

Common at BFSI/fintech companies — builds on what Synechron tests

Q21

Find Missing Number in Array 1 to N

Easy

An array contains N-1 elements from 1 to N with one number missing. Find it in $O(n)$ time and $O(1)$ space.

INPUT	OUTPUT
Input: [1,2,4,5,6], n=6	Output: 3

```
public class MissingNumber {
    // Sum formula: O(n) time, O(1) space
    public static int missing(int[] arr, int n) {
        long expected = (long) n * (n + 1) / 2;
        long actual = 0;
        for (int x : arr) actual += x;
        return (int)(expected - actual);
    }
    // XOR approach: same complexity, handles large numbers better
    public static int missingXOR(int[] arr, int n) {
        int xor = 0;
        for (int i = 1; i <= n; i++) xor ^= i;
        for (int x : arr) xor ^= x;
        return xor;
    }
}
```

Time: $O(n)$

Space: $O(1)$

Approach: Sum formula or XOR

Q22

Count Vowels, Consonants, Digits & Spaces

Easy

Count different character types in a string. Simple but frequently asked to test string traversal.

```
public class CharCount {
    public static void count(String s) {
        int vowels=0, cons=0, digits=0, spaces=0, special=0;
        String v = "aeiouAEIOU";
        for (char c : s.toCharArray()) {
            if (Character.isLetter(c)) {
```

```

if (v.indexOf(c) != -1) vowels++;
else cons++;
} else if (Character.isDigit(c)) digits++;
else if (c == ' ') spaces++;
else special++;
}
System.out.printf("Vowels:%d Consonants:%d Digits:%d Spaces:%d Special:%d\n",
vowels, cons, digits, spaces, special);
}
}

```

Time: O(n)

Space: O(1)

Approach: Character.isLetter/isDigit

Q23

GCD and LCM of Two Numbers

Easy

```

public class GCDAndLCM {
// Euclidean algorithm – most efficient
public static int gcd(int a, int b) {
while (b != 0) { int t = b; b = a % b; a = t; }
return a;
}
// LCM = (a * b) / gcd(a, b)
public static long lcm(int a, int b) {
return (long) a / gcd(a, b) * b; // divide first to prevent overflow
}
public static void main(String[] args) {
System.out.println("GCD(48,18) = " + gcd(48, 18)); // 6
System.out.println("LCM(4,6) = " + lcm(4, 6)); // 12
}
}

```

Time: O(log(min(a,b)))

Space: O(1)

Approach: Euclidean algorithm

Q24

Factorial — Iterative, Recursive & Large Numbers

Easy

```
import java.math.BigInteger;

public class Factorial {
    // Iterative – handles up to ~20! with long
    public static long factorial(int n) {
        if (n < 0) throw new IllegalArgumentException("Negative input");
        long result = 1;
        for (int i = 2; i <= n; i++) result *= i;
        return result;
    }
    // Recursive
    public static long factRecursive(int n) {
        if (n == 0 || n == 1) return 1;
        return n * factRecursive(n - 1);
    }
    // BigInteger – handles any size (100! etc.)
    public static BigInteger factBig(int n) {
        BigInteger result = BigInteger.ONE;
        for (int i = 2; i <= n; i++)
            result = result.multiply(BigInteger.valueOf(i));
        return result;
    }
    public static void main(String[] args) {
        System.out.println(factorial(10)); // 3628800
        System.out.println(factBig(50)); // huge number, no overflow
    }
}
```

Time: $O(n)$

Space: $O(1)$ iterative / $O(n)$ recursive

Approach: Simple loop or recursion

Q25

Reverse Words in a Sentence

Easy

Reverse the order of words in a sentence, not the characters. A common string problem that tests split + join knowledge.

INPUT	OUTPUT
Input: "Hello Java World"	Output: "World Java Hello"

```
public class ReverseWords {
```

```
public static String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        sb.append(words[i]);
        if (i > 0) sb.append(" ");
    }
    return sb.toString();
}

// One-liner using streams
public static String reverseWordsStream(String s) {
    String[] w = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = w.length-1; i >= 0; i--) sb.append(w[i]).append(" ");
    return sb.toString().trim();
}
}
```

Time: O(n)

Space: O(n)

Approach: Split + reverse loop

Q26

Abstract Class — Shape Area Calculator

Easy-Medium

A go-to OOP problem combining abstract classes, inheritance and polymorphism. Every Java OOP test at this level includes something like this.

```
import java.util.Scanner;

abstract class Shape {
    String color;
    Shape(String color) { this.color = color; }
    abstract double area();
    abstract double perimeter();
    public void printInfo() {
        System.out.printf("%s [%s] area=%.2f perimeter=%.2f%n",
            getClass().getSimpleName(), color, area(), perimeter());
    }
}

class Circle extends Shape {
    double r;
    Circle(String c, double r) { super(c); this.r = r; }
    double area() { return Math.PI * r * r; }
    double perimeter() { return 2 * Math.PI * r; }
}

class Rectangle extends Shape {
    double l, w;
    Rectangle(String c, double l, double w) { super(c); this.l=l; this.w=w; }
    double area() { return l * w; }
    double perimeter() { return 2 * (l + w); }
}

class Triangle extends Shape {
    double a, b, c;
    Triangle(String col, double a, double b, double c) {
        super(col); this.a=a; this.b=b; this.c=c;
    }
    double area() { // Heron's formula
        double s = (a+b+c)/2;
        return Math.sqrt(s*(s-a)*(s-b)*(s-c));
    }
    double perimeter() { return a + b + c; }
}

public class ShapeDemo {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle("Red", 5),
            new Rectangle("Blue", 4, 6),
```



```

new Triangle("Green", 3, 4, 5)
};
for (Shape s : shapes) s.printInfo();
}
}

```

Time: O(1) per shape

Space: O(1)

Approach: Polymorphism + abstract class

Q27

Custom Exception — Bank Account

Easy-Medium

Combines OOP + Exception Handling. Very common combined problem in Synchro-level tests. Demonstrates custom exception design and proper throws/catch usage.

```

class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String msg) { super(msg); }
}

class BankAccount {
    private String owner;
    private double balance;

    BankAccount(String owner, double balance) {
        this.owner = owner; this.balance = balance;
    }

    void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("Invalid amount");
        balance += amt;
        System.out.printf("Deposited %.2f. Balance: %.2f\n", amt, balance);
    }

    void withdraw(double amt) throws InsufficientFundsException {
        if (amt <= 0) throw new IllegalArgumentException("Invalid amount");
        if (amt > balance)
            throw new InsufficientFundsException(
                "Need Rs " + (amt - balance) + " more");
        balance -= amt;
        System.out.printf("Withdrew %.2f. Balance: %.2f\n", amt, balance);
    }

    public String toString() {
        return owner + "'s account: Rs " + balance;
    }
}

public class BankDemo {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Nitesh", 5000);
        try {
            acc.deposit(2000);

```

```
acc.withdraw(3000);  
acc.withdraw(10000); // throws  
} catch (InsufficientFundsException e) {  
    System.out.println("Error: " + e.getMessage());  
} finally {  
    System.out.println(acc);  
}  
}  
}
```

Q28

Sort Array of Strings by Length then Alphabetically

Easy

Sort a string array first by length (ascending), then alphabetically for equal lengths. Tests Comparator knowledge — frequently asked to demonstrate lambda skills.

INPUT	OUTPUT
Input: ["banana", "fig", "apple", "kiwi", "pear"]	Output: [fig, kiwi, pear, apple, banana]

```
import java.util.*;

public class SortStrings {
    public static void main(String[] args) {
        String[] arr = {"banana", "fig", "apple", "kiwi", "pear"};
        // Sort by length, then alphabetically for ties
        Arrays.sort(arr, (a, b) -> {
            if (a.length() != b.length()) return a.length() - b.length();
            return a.compareTo(b);
        });
        System.out.println(Arrays.toString(arr));
        // Using Comparator.comparing – cleaner
        Arrays.sort(arr,
            Comparator.comparingInt(String::length)
                .thenComparing(Comparator.naturalOrder())
        );
        System.out.println(Arrays.toString(arr));
    }
}
```

Time: $O(n \log n)$

Space: $O(1)$

Approach: Comparator with lambda

Q29

Intersection and Union of Two Arrays

Easy

Find elements common to both arrays (intersection) and elements in either array (union). Directly tests HashSet knowledge.

```
import java.util.*;

public class SetOperations {
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4, 5};
        int[] b = {3, 4, 5, 6, 7};
```

```

HashSet<Integer> setA = new HashSet<>();
for (int x : a) setA.add(x);
// Intersection
HashSet<Integer> inter = new HashSet<>(setA);
for (int x : b) {
    // Keep only elements that are in setA
}
inter.retainAll(new HashSet<>(Arrays.asList(
    Arrays.stream(b).boxed().toArray(Integer[]::new))););
System.out.println("Intersection: " + new TreeSet<>(inter));
// Union
HashSet<Integer> union = new HashSet<>(setA);
for (int x : b) union.add(x);
System.out.println("Union: " + new TreeSet<>(union));
// Simpler approach
HashSet<Integer> i2 = new HashSet<>();
for (int x : b) if (setA.contains(x)) i2.add(x);
System.out.println("Intersection v2: " + new TreeSet<>(i2));
}
}

```

Time: $O(n + m)$

Space: $O(n)$

Approach: HashSet contains

Q30

Matrix Transpose & Diagonal Sum

Easy-Medium

Matrix problems appear in fresher tests. Know transpose (flip rows and cols) and diagonal sum (sum of main and anti-diagonal).

```
public class MatrixOps {  
    // Transpose: matrix[i][j] -> matrix[j][i]  
    public static int[][] transpose(int[][] m) {  
        int rows = m.length, cols = m[0].length;  
        int[][] result = new int[cols][rows];  
        for (int i = 0; i < rows; i++)  
            for (int j = 0; j < cols; j++)  
                result[j][i] = m[i][j];  
        return result;  
    }  
  
    // Sum of both diagonals (for square matrix)  
    public static int diagonalSum(int[][] m) {  
        int n = m.length, sum = 0;  
        for (int i = 0; i < n; i++) {  
            sum += m[i][i]; // main diagonal  
            sum += m[i][n - 1 - i]; // anti-diagonal  
        }  
        // If n is odd, center element counted twice – subtract it  
        if (n % 2 == 1) sum -= m[n/2][n/2];  
        return sum;  
    }  
  
    public static void printMatrix(int[][] m) {  
        for (int[] row : m) {  
            for (int x : row) System.out.printf("%4d", x);  
            System.out.println();  
        }  
    }  
  
    public static void main(String[] args) {  
        int[][] mat = {{1,2,3},{4,5,6},{7,8,9}};  
        System.out.println("Original:");  
        printMatrix(mat);  
        System.out.println("Transposed:");  
        printMatrix(transpose(mat));  
        System.out.println("Diagonal sum: " + diagonalSum(mat)); // 25  
    }  
}
```

Time: $O(n^2)$

Space: $O(n^2)$

Approach: Nested loops

SECTION 4: QUICK CODE REFERENCE

Shorter solutions for common patterns — know these cold

Q31 Check Armstrong Number

A number is Armstrong if sum of its digitsⁿ == the number (n = digit count). E.g., $153 = 1^3 + 5^3 + 3^3$

```
public static boolean isArmstrong(int num) {
    String s = String.valueOf(num);
    int n = s.length(), sum = 0;
    for (char c : s.toCharArray())
        sum += (int) Math.pow(c - '0', n);
    return sum == num;
}

// isArmstrong(153) -> true | isArmstrong(370) -> true
```

Q32 Swap Two Numbers Without Temp Variable

Swap using XOR or arithmetic. No extra variable allowed.

```
// XOR swap (no overflow risk)
int a = 5, b = 10;
a = a ^ b; // a = 15 (XOR)
b = a ^ b; // b = 5 (original a)
a = a ^ b; // a = 10 (original b)
System.out.println("a=" + a + " b=" + b); // a=10 b=5

// Arithmetic swap (may overflow for large numbers)
a = a + b; b = a - b; a = a - b;
```

Q33 Print Pattern — Number Triangle

Print a right-angled triangle pattern with numbers. Pattern printing tests loop mastery.

```
// Pattern for n=4:
// 1
// 1 2
// 1 2 3
// 1 2 3 4

public static void printTriangle(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
        }
        System.out.println();
    }
}
```

Q34 Count Occurrences of Character in String

Count how many times a specific character appears in a string.

```
public static int countChar(String s, char target) {
    int count = 0;
    for (char c : s.toCharArray())
        if (c == target) count++;
    return count;
}

// One-liner with streams:
long cnt = s.chars().filter(c -> c == target).count();
// countChar("banana", 'a') -> 3
```

Q35 Find Largest & Smallest in Array

Find max and min elements in a single traversal without sorting.

```
public static void findMinMax(int[] arr) {
    int min = arr[0], max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] < min) min = arr[i];
        if (arr[i] > max) max = arr[i];
    }
    System.out.println("Min: " + min + " Max: " + max);
}

// O(n) time, O(1) space
// Also: Arrays.stream(arr).min().getAsInt()
```

FINAL: TOP 8 PROBLEMS TO KNOW COLD

These are the most likely to appear — practice until you write them in under 5 minutes

★ 1	First Non-Repeating Character	LinkedHashMap freq → find first with value 1	Q1
★ 2	Palindrome Check (String)	Two pointer: left++ right-- compare chars	Q2
★ 3	Reverse String (no reverse())	char[] two-pointer swap	Q3
★ 4	Pair Sum to Target	HashSet: for each num, check if (target-num) in set	Q4
★ 5	Primes in Range (N largest + sum)	isPrime() + ArrayList + sort descending	Q5
★ 6	Even Left / Odd Right Partition	Two-pointer: swap arr[left] odd with arr[right] even	Q6
★ 7	Runtime Polymorphism (code + output)	Animal a = new Dog(); a.sound() → Dog's method	Q7
★ 8	Diamond Problem + Interface solution	Multiple class inheritance banned → use interfaces	Q8

AM.

SECTION 5: COLLECTIONS FRAMEWORK PROBLEMS

ArrayList · HashMap · HashSet · LinkedList — must-know for Synchro

Q36

Group Anagrams Together

Medium

Given an array of strings, group all anagrams together. Combines HashMap with sorting — a classic collections + string problem.

INPUT	OUTPUT
Input: ["eat", "tea", "tan", "ate", "nat", "bat"]	Groups: [eat, tea, ate] [tan, nat] [bat]

```
import java.util.*;

public class GroupAnagrams {

    public static Map<String, List<String>> group(String[] words) {
        // KEY IDEA: anagrams have the same sorted characters
        HashMap<String, List<String>> map = new HashMap<>();
        for (String word : words) {
            // Sort the word's characters to form a canonical key
            char[] chars = word.toLowerCase().toCharArray();
            Arrays.sort(chars);
            String key = new String(chars); // e.g. "eat" -> "aet", "tea" -> "aet"
            // Group words with the same key
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
        }
        return map;
    }

    public static void main(String[] args) {
        String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
        Map<String, List<String>> result = group(words);
        for (List<String> grp : result.values()) {
            System.out.println(grp);
        }
    }
}

// Output:
// [eat, tea, ate]
// [tan, nat]
// [bat]
```

Time: $O(n * k \log k)$

Space: $O(n * k)$

Approach: k = avg word length

Q37

Top K Most Frequent Elements

Medium

Given an array of integers, return the K most frequently occurring elements. Tests HashMap + PriorityQueue together — a fintech favourite.

INPUT	OUTPUT
Input: [1,1,1,2,2,3], k=2	Output: [1, 2]
Input: [4,4,5,5,5,6], k=1	Output: [5]

```
import java.util.*;

public class TopKFrequent {

    public static List<Integer> topK(int[] nums, int k) {
        // Step 1: Count frequencies
        HashMap<Integer, Integer> freq = new HashMap<>();
        for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);
        // Step 2: Min-heap of size k (keeps k largest frequencies)
        // Comparator: sort by frequency ascending (min at top)
        PriorityQueue<Integer> pq = new PriorityQueue<>((a, b) -> freq.get(a) - freq.get(b));
        for (int num : freq.keySet()) {
            pq.offer(num);
            if (pq.size() > k) pq.poll(); // remove least frequent
        }
        // Step 3: Collect result
        List<Integer> result = new ArrayList<>(pq);
        result.sort((a, b) -> freq.get(b) - freq.get(a)); // sort by freq desc
        return result;
    }

    // Simpler approach: sort HashMap entries by value
    public static List<Integer> topKSimple(int[] nums, int k) {
        HashMap<Integer, Integer> freq = new HashMap<>();
        for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);
        List<Map.Entry<Integer, Integer>> list = new ArrayList<>(freq.entrySet());
        list.sort((a, b) -> b.getValue() - a.getValue()); // sort by freq desc
        List<Integer> result = new ArrayList<>();
        for (int i = 0; i < k; i++) result.add(list.get(i).getKey());
        return result;
    }

    public static void main(String[] args) {
        int[] nums = {1,1,1,2,2,3};
        System.out.println(topKSimple(nums, 2)); // [1, 2]
    }
}
```

```
}
```

Time: $O(n \log k)$

Space: $O(n)$

Approach: HashMap + PriorityQueue

Q38

Implement a Student Marks Tracker using Collections

Easy-Medium

A real-world Collections problem: store students and marks in a HashMap, find toppers, compute class average, and sort students by marks. This type of problem directly tests practical Collections usage.

```
import java.util.*;

public class StudentTracker {

    public static void main(String[] args) {
        // Store student -> marks
        LinkedHashMap<String, Integer> marks = new LinkedHashMap<>();
        marks.put("Alice", 92);
        marks.put("Bob", 78);
        marks.put("Charlie", 85);
        marks.put("Dave", 92);
        marks.put("Eve", 70);

        // 1. Print all students in insertion order
        System.out.println("All students:");
        marks.forEach((name, score) ->
            System.out.printf(" %-10s %d\n", name, score));

        // 2. Class average
        double avg = marks.values().stream()
            .mapToInt(Integer::intValue)
            .average()
            .orElse(0);
        System.out.printf("Class average: %.2f\n", avg);

        // 3. Topper(s) – handle ties
        int maxScore = Collections.max(marks.values());
        System.out.print("Topper(s): ");
        marks.entrySet().stream()
            .filter(e -> e.getValue() == maxScore)
            .forEach(e -> System.out.print(e.getKey() + " "));
        System.out.println("(" + maxScore + ")");

        // 4. Sort by marks descending
        System.out.println("Ranked:");
        marks.entrySet().stream()
            .sorted((a, b) -> b.getValue() - a.getValue())
            .forEach(e -> System.out.printf(" %-10s %d\n",
                e.getKey(), e.getValue()));

        // 5. Students who passed (marks >= 80)
        System.out.print("Passed: ");
        marks.entrySet().stream()
            .filter(e -> e.getValue() >= 80)
            .map(Map.Entry::getKey)
            .forEach(n -> System.out.print(n + " "));
    }
}
```

```
}  
}
```

Time: $O(n \log n)$

Space: $O(n)$

Approach: Stream filter/sort on HashMap

Q39

Two-Sum Using HashMap — Return Indices

Easy

Given an array and a target, return the INDICES of two numbers that add up to the target. Variant of the pair-sum problem — asks for indices, not values. Classic $O(n)$ HashMap trick.

INPUT	OUTPUT
Input: [2,7,11,15], target=9	Output: [0, 1] (2+7=9)
Input: [3,2,4], target=6	Output: [1, 2] (2+4=6)

```
import java.util.*;  
  
public class TwoSumIndices {  
    public static int[] twoSum(int[] nums, int target) {  
        // Map: value -> index  
        HashMap<Integer, Integer> map = new HashMap<>();  
  
        for (int i = 0; i < nums.length; i++) {  
            int complement = target - nums[i];  
  
            // If complement already seen, we found our pair  
            if (map.containsKey(complement)) {  
                return new int[]{map.get(complement), i};  
            }  
  
            // Store current value and its index  
            map.put(nums[i], i);  
        }  
  
        return new int[]{-1, -1}; // no pair found  
    }  
  
    public static void main(String[] args) {  
        int[] result = twoSum(new int[]{2,7,11,15}, 9);  
        System.out.println(Arrays.toString(result)); // [0, 1]  
    }  
}  
  
// WHY HashMap: complement lookup is  $O(1)$  average  
// Single pass through array -> overall  $O(n)$ 
```

Time: $O(n)$

Space: $O(n)$

Approach: HashMap value->index lookup

SECTION 6: EXCEPTION HANDLING PROBLEMS

try-catch-finally · Custom Exceptions · Common pitfalls

Q40

Safe Integer Parser with Custom Exception

Easy

Read an integer from user input safely — handle all exceptions: `NumberFormatException`, `NullPointerException`, `ArithmeticException`. Demonstrates practical exception handling in a real input scenario.

```
import java.util.Scanner;

class InvalidInputException extends RuntimeException {
    public InvalidInputException(String msg) { super(msg); }
}

public class SafeParser {
    public static int safeParseInt(String input) {
        if (input == null || input.trim().isEmpty())
            throw new InvalidInputException("Input cannot be null or empty");
        try {
            return Integer.parseInt(input.trim());
        } catch (NumberFormatException e) {
            throw new InvalidInputException("Not a valid integer: '" + input + "'");
        }
    }

    public static int divide(int a, int b) {
        try {
            return a / b;
        } catch (ArithmeticException e) {
            System.out.println("Division by zero caught: " + e.getMessage());
            return 0;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] inputs = {"42", "abc", "", null, "100"};
        for (String inp : inputs) {
            try {
                int val = safeParseInt(inp);
                System.out.println("Parsed: " + val);
            } catch (InvalidInputException e) {
                System.out.println("Error: " + e.getMessage());
            } finally {
                System.out.println("--- attempt done ---");
            }
        }
    }
}
```

```

System.out.println(divide(10, 0)); // 0 (handled)
System.out.println(divide(10, 2)); // 5
sc.close();
}
}

```

Time: O(n)

Space: O(1)

Approach: try-catch-finally per input

NOTE: finally always runs — even when an exception is thrown or return is called. Use it to close resources or log.

Q41

Exception Handling Output Prediction (MCQ-style)

Easy

Synechron and similar tests ask you to predict what a try-catch-finally block prints. Know the exact execution order cold.

```

// PREDICT THE OUTPUT – common interview question pattern
// Snippet 1:
try {
    System.out.println("try");
    int x = 1 / 0; // ArithmeticException thrown here
    System.out.println("after"); // NEVER REACHED
} catch (ArithmeticException e) {
    System.out.println("catch");
} finally {
    System.out.println("finally");
}
// OUTPUT: try -> catch -> finally

// Snippet 2: return inside try
public static String test() {
    try {
        System.out.println("try");
        return "from try";
    } finally {
        System.out.println("finally"); // STILL runs before return!
    }
}
// OUTPUT: try -> finally -> returns "from try"

// Snippet 3: catch re-throws
try {
    int[] a = new int[3];
    System.out.println(a[5]); // ArrayIndexOutOfBoundsException
} catch (NullPointerException e) {
    System.out.println("NPE"); // NOT caught here – wrong type
} catch (Exception e) {
    System.out.println("General: " + e.getClass().getSimpleName());
}

```

```
} finally {  
    System.out.println("done");  
}  
// OUTPUT: General: ArrayIndexOutOfBoundsException -> done  
// Snippet 4: Multi-catch  
try {  
    String s = null;  
    s.length(); // NullPointerException  
} catch (NullPointerException | IllegalArgumentException e) {  
    System.out.println("Caught: " + e.getClass().getSimpleName());  
}  
// OUTPUT: Caught: NullPointerException
```

TIP: Key rule: catch blocks are checked top-to-bottom. Always put specific exceptions before general Exception.
finally ALWAYS runs.

Q42

Multiple Exception Handling — File & Number Operations

Easy-Medium

A comprehensive exception handling program that demonstrates multi-catch, checked vs unchecked exceptions, and chained exception patterns.

```
import java.util.*;

public class MultiException {
    // Method that may throw multiple exceptions
    public static int processInput(String[] args, int index) throws Exception {
        // Might throw ArrayIndexOutOfBoundsException
        String input = args[index];

        // Might throw NumberFormatException
        int num = Integer.parseInt(input);

        // Might throw ArithmeticException
        int result = 100 / num;

        return result;
    }

    public static void main(String[] args) {
        // Test with various inputs
        String[][] testCases = {
            {"5"}, // valid
            {"0"}, // division by zero
            {"abc"}, // number format
            {} // no args – index out of bounds
        };

        for (String[] tc : testCases) {
            try {
                int res = processInput(tc, 0);
                System.out.println("Result: " + res);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("No argument provided");
            } catch (NumberFormatException e) {
                System.out.println("Invalid number: " + e.getMessage());
            } catch (ArithmeticException e) {
                System.out.println("Math error: " + e.getMessage());
            } catch (Exception e) {
                System.out.println("Unexpected: " + e.getMessage());
            }
        }

        // Output:
        // Result: 20
        // Math error: / by zero
    }
}
```

```
// Invalid number: For input string: "abc"  
// No argument provided
```

SECTION 7: DSA PATTERN PROBLEMS

Two-pointer · Stack · Queue · Sliding Window · Recursion

Q43

Reverse a Linked List — Iterative & Recursive

Easy-Medium

Reversing a linked list is a fundamental pointer manipulation problem. Know both iterative (3 pointers) and recursive approaches.

INPUT	OUTPUT
Input: 1 -> 2 -> 3 -> 4 -> 5 -> null	Output: 5 -> 4 -> 3 -> 2 -> 1 -> null

```
class Node {
    int data; Node next;
    Node(int d) { data = d; next = null; }
}

public class ReverseLinkedList {
    // Iterative - O(n), O(1) space - PREFERRED
    public static Node reverse(Node head) {
        Node prev = null;
        Node curr = head;
        Node next = null;
        while (curr != null) {
            next = curr.next; // save next
            curr.next = prev; // reverse the pointer
            prev = curr; // move prev forward
            curr = next; // move curr forward
        }
        return prev; // prev is now the new head
    }

    // Recursive - O(n), O(n) stack space
    public static Node reverseRec(Node head) {
        if (head == null || head.next == null) return head;
        Node newHead = reverseRec(head.next); // recurse to end
        head.next.next = head; // make next node point back to current
        head.next = null; // cut forward link
        return newHead;
    }

    public static void print(Node head) {
        for (Node n = head; n != null; n = n.next)
            System.out.print(n.data + (n.next != null ? " -> " : " -> null"));
        System.out.println();
    }
}
```

```

public static void main(String[] args) {
    Node head = new Node(1);
    head.next = new Node(2); head.next.next = new Node(3);
    head.next.next.next = new Node(4);
    head.next.next.next.next = new Node(5);
    print(head); // 1 -> 2 -> 3 -> 4 -> 5 -> null
    head = reverse(head);
    print(head); // 5 -> 4 -> 3 -> 2 -> 1 -> null
}
}

```

Time: O(n)

Space: O(1) iterative / O(n) recursive

Approach: 3-pointer reversal

Q44

Stack — Evaluate Postfix Expression

Medium

Evaluate a postfix (Reverse Polish Notation) expression using a Stack. Classic stack application — very commonly asked to test stack + string parsing.

INPUT	OUTPUT
Input: "2 3 4 * +" (= 2 + 3*4)	Output: 14
Input: "5 1 2 + 4 * + 3 -"	Output: 14

```

import java.util.*;

public class PostfixEvaluator {
    public static int evaluate(String expr) {
        Deque<Integer> stack = new ArrayDeque<>();
        String[] tokens = expr.trim().split("\\s+");
        for (String token : tokens) {
            // If it's an operator, pop two operands and apply
            if (token.equals("+") || token.equals("-") ||
                token.equals("*") || token.equals("/")) {
                int b = stack.pop(); // second operand (popped first)
                int a = stack.pop(); // first operand
                switch (token) {
                    case "+": stack.push(a + b); break;
                    case "-": stack.push(a - b); break;
                    case "*": stack.push(a * b); break;
                    case "/": stack.push(a / b); break;
                }
            } else {
                // It's a number — push onto stack
                stack.push(Integer.parseInt(token));
            }
        }
    }
}

```

```
}  
return stack.pop(); // final result  
}  
  
public static void main(String[] args) {  
    System.out.println(evaluate("2 3 4 * +")); // 14  
    System.out.println(evaluate("5 1 2 + 4 * + 3 -")); // 14  
}  
}
```

Time: $O(n)$

Space: $O(n)$

Approach: Stack push/pop per token

Q45

Queue — Generate Binary Numbers 1 to N

Easy-Medium

Generate binary representations of numbers 1 to N using a Queue. Elegant BFS-style queue problem — tests Queue operations and binary number knowledge.

INPUT	OUTPUT
Input: N = 5	1 10 11 100 101

```
import java.util.*;

public class BinaryNumbers {
    // IDEA: start with "1", then for each dequeued number s,
    // enqueue s+"0" and s+"1" – BFS generates in order
    public static void generateBinary(int n) {
        Queue<String> queue = new LinkedList<>();
        queue.offer("1"); // start with "1"

        for (int i = 0; i < n; i++) {
            String current = queue.poll();
            System.out.println(current);
            // Enqueue children
            queue.offer(current + "0");
            queue.offer(current + "1");
        }
    }

    // Alternative: Integer.toBinaryString()
    public static void generateBinarySimple(int n) {
        for (int i = 1; i <= n; i++)
            System.out.println(Integer.toBinaryString(i));
    }

    public static void main(String[] args) {
        generateBinary(5);
        // 1, 10, 11, 100, 101
    }
}
```

Time: O(n)

Space: O(n)

Approach: Queue BFS-style

Q46

Sliding Window — Maximum Sum Subarray of Size K

Medium

Find the maximum sum of any contiguous subarray of size exactly K. Classic sliding window — add new element, remove old element, $O(n)$ instead of $O(n*k)$.

INPUT	OUTPUT
Input: [2,1,5,1,3,2], k=3	Max sum = 9 (subarray [5,1,3])
Input: [2,3,4,1,5], k=2	Max sum = 7 (subarray [3,4])

```
public class SlidingWindowMaxSum {  
    public static int maxSum(int[] arr, int k) {  
        int n = arr.length;  
        if (n < k) return -1;  
        // Compute sum of first window  
        int windowSum = 0;  
        for (int i = 0; i < k; i++) windowSum += arr[i];  
        int maxSum = windowSum;  
        // Slide the window: add next element, remove first element  
        for (int i = k; i < n; i++) {  
            windowSum += arr[i]; // add new element (right)  
            windowSum -= arr[i - k]; // remove old element (left)  
            maxSum = Math.max(maxSum, windowSum);  
        }  
        return maxSum;  
    }  
    // Also print the subarray with max sum  
    public static void maxSumSubarray(int[] arr, int k) {  
        int windowSum = 0;  
        for (int i = 0; i < k; i++) windowSum += arr[i];  
        int maxSum = windowSum, start = 0;  
        for (int i = k; i < arr.length; i++) {  
            windowSum += arr[i] - arr[i - k];  
            if (windowSum > maxSum) {  
                maxSum = windowSum;  
                start = i - k + 1;  
            }  
        }  
        System.out.println("Max sum: " + maxSum);  
        System.out.println("Subarray: " + java.util.Arrays.toString(  
            java.util.Arrays.copyOfRange(arr, start, start + k)));  
    }  
    public static void main(String[] args) {  
        int[] arr = {2, 1, 5, 1, 3, 2};  
        System.out.println(maxSum(arr, 3)); // 9  
    }  
}
```

```
maxSumSubarray(arr, 3);  
}  
}
```

Time: $O(n)$

Space: $O(1)$

Approach: Fixed-size sliding window

Q47

Recursion — Power Set / All Subsets of Array

Medium

Generate all possible subsets (power set) of a given array using recursion. Tests recursive thinking and ArrayList manipulation.

INPUT	OUTPUT
Input: [1, 2, 3]	[1] [2] [1,2] [3] [1,3] [2,3] [1,2,3]

```
import java.util.*;

public class Subsets {
    // Recursive backtracking
    public static void generateSubsets(int[] arr, int index,
        List<Integer> current,
        List<List<Integer>> result) {
        // Base case: add current subset to result
        result.add(new ArrayList<>(current));

        for (int i = index; i < arr.length; i++) {
            current.add(arr[i]); // include arr[i]
            generateSubsets(arr, i + 1, current, result); // recurse
            current.remove(current.size() - 1); // backtrack
        }
    }

    // Iterative using bit masking
    public static void subsetsIterative(int[] arr) {
        int n = arr.length;
        int total = 1 << n; // 2^n subsets
        for (int mask = 0; mask < total; mask++) {
            System.out.print("[");
            for (int i = 0; i < n; i++) {
                if ((mask & (1 << i)) != 0)
                    System.out.print(arr[i] + " ");
            }
            System.out.println("");
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        List<List<Integer>> result = new ArrayList<>();
    }
}
```

```
generateSubsets(arr, 0, new ArrayList<>(), result);  
result.forEach(System.out::println);  
}  
}
```

Time: $O(2^n * n)$

Space: $O(2^n)$

Approach: Backtracking recursion

SECTION 8: JAVA CONCEPT MCQs

Objective questions on OOP, Collections, Exceptions — know the correct answer AND why

HackerRank tests often include MCQ sections alongside coding. These are the most commonly tested Java conceptual questions.

Q48. What is the output of: `System.out.println(10 + 20 + "Hello" + 10 + 20);`

- A) 30Hello30
- ✓ B) 30Hello1020
- C) 1020Hello1020
- D) Compile Error

Explanation: Java evaluates left to right. $10+20=30$ (both int), then $30+"Hello"="30Hello"$ (String concat), then $"30Hello"+10="30Hello10"$, then $"30Hello10"+20="30Hello1020"$.

Q49. Which collection maintains insertion order AND allows duplicates?

- A) HashSet
- B) TreeSet
- ✓ C) ArrayList
- D) HashMap

Explanation: ArrayList maintains insertion order and allows duplicate elements. HashSet/TreeSet don't allow duplicates. HashMap is key-value, not a simple list.

Q50. What is the difference between `==` and `.equals()` for String comparison?

- A) No difference
- ✓ B) `==` compares reference, `.equals()` compares content
- C) `==` compares content, `.equals()` compares reference
- D) Both compare content

Explanation: String s1="hello"; String s2=new String("hello"); s1==s2 is FALSE (different objects) but s1.equals(s2) is TRUE (same content). Always use .equals() for String comparison.

Q51. What is the output? `try { return 1; } finally { return 2; }`

- A) 1
- ✓ B) 2
- C) Compile Error
- D) Runtime Exception

Explanation: finally block always executes — even when return is in try. The finally return (2) overrides the try return (1). Avoid return in finally — it suppresses exceptions too.

Q52. Which access modifier makes a member accessible ONLY within the same class?

- A) default (no modifier)
- B) protected
- C) public
- ✓ D) private

Explanation: private: same class only. default: same package. protected: same package + subclasses. public: everywhere.

Q53. What happens when you try to store a duplicate key in a HashMap?

- A) Exception thrown

B) Old value kept, new value ignored

✓ C) New value overwrites old value

D) Both values stored

Explanation: HashMap.put(key, value) — if key already exists, the old value is replaced with the new value. put() returns the OLD value that was replaced (or null if new key).

Q54. What is the output of: `int[] a = {1,2,3}; System.out.println(a);`

A) [1, 2, 3]

B) 1 2 3

✓ C) [I@hashcode (memory address)

D) Compile Error

Explanation: Arrays don't override toString(). Printing an array directly gives its type ([I for int[]) and memory hash. Use Arrays.toString(a) to print [1, 2, 3].

Q55. Which of these is NOT a valid way to create a String in Java?

A) String s = "hello";

B) String s = new String("hello");

C) String s = String.valueOf(123);

✓ D) String s = new String;

Explanation: new String without parentheses is invalid syntax. A) string literal (uses pool), B) new String object (heap), C) valueOf() converts 123 to "123" — all valid.

Q56. What is the time complexity of ArrayList.get(index)?

A) O(n)

B) O(log n)

✓ C) O(1)

D) O(n log n)

Explanation: ArrayList is backed by an array, so random access by index is O(1) — direct memory address calculation. LinkedList.get(index) is O(n) — must traverse from head.

Q57. Can an abstract class have a constructor?

A) No — abstract classes cannot have constructors

✓ B) Yes — but it can only be called via super() from a subclass

C) Yes — and it can be called directly with new AbstractClass()

D) Only if it has no abstract methods

Explanation: Abstract classes CAN have constructors. They are called when a subclass is instantiated via super(). You cannot do new AbstractClass() directly — that's a compile error.

Q58. What is autoboxing in Java?

A) Automatic type casting between int and long

✓ B) Automatic conversion between primitive types and their wrapper classes

C) Wrapping exceptions in try-catch automatically

D) Automatic garbage collection

Explanation: Autoboxing: int -> Integer automatically. Unboxing: Integer -> int automatically. Example: List list = new ArrayList<>(); list.add(5); — the int 5 is autoboxed to Integer.

Q59. What does the 'static' keyword mean for a method?

A) Method cannot be overridden

✓ B) Method belongs to the class, not to any instance

C) Method runs at compile time

D) Method is thread-safe

Explanation: static methods belong to the class itself. Called as `ClassName.method()` — no object needed. Cannot access instance variables (`this`) inside static methods. `main()` is static — JVM calls it without creating an object.

SECTION 9: BONUS — SHORT ANSWER PROBLEMS

5 more problems with compact solutions — maximise your preparation

Q60

Print Star Pattern — Diamond Shape

Easy

Print a diamond pattern of stars for given n (half-height).

```
public static void diamond(int n) {
    // Upper half (including middle)
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < n - i; j++) System.out.print(" ");
        for (int j = 0; j < 2*i-1; j++) System.out.print("*");
        System.out.println();
    }
    // Lower half
    for (int i = n-1; i >= 1; i--) {
        for (int j = 0; j < n - i; j++) System.out.print(" ");
        for (int j = 0; j < 2*i-1; j++) System.out.print("*");
        System.out.println();
    }
}

// n=3 output:
// *
// ***
// *****
// ***
// *
```

Q61

Check if Array is Sorted

Easy

Check if an integer array is sorted in ascending order in a single pass.

```
public static boolean isSorted(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] < arr[i-1]) return false;
    }
    return true;
}

// isSorted(new int[]{1,2,3,4,5}) -> true
// isSorted(new int[]{1,3,2,4,5}) -> false
```

Q62

Sum of Digits Until Single Digit (Digital Root)

Easy

Repeatedly sum digits of a number until you get a single digit.

```
public static int digitalRoot(int n) {
    while (n >= 10) {
        int sum = 0;
        while (n > 0) { sum += n % 10; n /= 10; }
        n = sum;
    }
    return n;
}

// digitalRoot(493) -> 4+9+3=16 -> 1+6=7
// One-line formula: n==0 ? 0 : (n%9==0 ? 9 : n%9)
```

Q63

Count Words Starting With a Vowel

Easy

Count how many words in a sentence start with a vowel (a,e,i,o,u).

```
public static int countVowelWords(String sentence) {
    String vowels = "aeiouAEIOU";
    int count = 0;
    for (String word : sentence.trim().split("\\s+")) {
        if (!word.isEmpty() && vowels.indexOf(word.charAt(0)) != -1)
            count++;
    }
    return count;
}

// "Apple is an amazing fruit" -> 3 (Apple, is->no, an, amazing)
// "Every orange is excellent" -> 3
```

Q64

Implement Interface — Comparable for Custom Sorting

Easy-Medium

Implement Comparable in a class to enable natural ordering with Collections.sort().

```
import java.util.*;

class Employee implements Comparable<Employee> {
    String name;
    int salary;
```

```
Employee(String name, int salary) {
    this.name = name; this.salary = salary;
}

@Override
public int compareTo(Employee other) {
    // Sort by salary ascending; name ascending for ties
    if (this.salary != other.salary) return this.salary - other.salary;
    return this.name.compareTo(other.name);
}

public String toString() { return name + "(" + salary + ")"; }
}

// Usage
List<Employee> emp = new ArrayList<>();
emp.add(new Employee("Zara", 50000));
emp.add(new Employee("Alice", 70000));
emp.add(new Employee("Bob", 50000));
Collections.sort(emp);
System.out.println(emp); // [Bob(50000), Zara(50000), Alice(70000)]
```

MASTER CHEAT SHEET — READ THIS DAY BEFORE THE TEST

INPUT TEMPLATE

```
import java.util.*;

Scanner sc = new Scanner(System.in);

int n = sc.nextInt(); // read int

sc.nextLine(); // flush newline after nextInt

String line = sc.nextLine(); // read whole line

String[] parts = line.split(" "); // split by space

int x = Integer.parseInt(parts[0]); // String to int
```

MOST USED PATTERNS

```
Freq count: map.getOrDefault(key, 0) + 1

Two-pointer: left=0, right=n-1, while(left<right)

Sliding window: windowSum += arr[i] - arr[i-k]

Sort descending: (a,b)->b-a OR Collections.reverseOrder()

String clean: s.toLowerCase().replaceAll("[^a-z]", "")

Char to int: c - '0' (e.g., '5'-'0' = 5)

isPrime check up to sqrt(n): i*i <= n
```

COLLECTIONS ONE-LINERS

```
List from array: Arrays.asList(1,2,3)

Sort list: Collections.sort(list)

Max in list: Collections.max(list)

Reverse list: Collections.reverse(list)

Array to string: Arrays.toString(arr)

Fill array: Arrays.fill(arr, 0)

Copy array: Arrays.copyOf(arr, len)
```

KEY EXCEPTIONS TO KNOW

```
NullPointerException – calling method on null

ArrayIndexOutOfBoundsException – arr[-1] or arr[arr.length]

NumberFormatException – Integer.parseInt("abc")

ArithmeticException – division by zero (int/int only)

ClassCastException – (Dog) new Cat()

StackOverflowError – infinite recursion
```

Wednesday 20 May · Reach by 9:30 AM · Hinjewadi Phase 3, 1st Floor